



Proyecto #2 Valor 2%

Entrega: Semana 9

1) Temas de Estudio

1. Tema 1 Introducción al lenguaje de programación.
2. Tema 2 Particularidades del lenguaje C#.
3. Tema 3 Manejo de excepciones en C#
4. Tema 4 Conceptos de Interfaz Gráfica
5. Tema 5 Subprocesamiento múltiple
6. Tema 6 Bases de datos y componentes ADO.NET
7. Tema 7 Colecciones
8. Tema 8 Redes

2) Objetivo

Desarrollar la capacidad de integrar subprocesamiento múltiple, colecciones, acceso a bases de datos con ADO.NET y comunicación por redes en aplicaciones cliente/servidor construidas en C#, orientadas a la gestión eficiente y concurrente de la información empresarial.

3) Software de Desarrollo

Visual Studio 2026, utilizando el lenguaje de programación C#. El proyecto se implementará bajo un enfoque de programación orientada a objetos para favorecer el diseño modular y reutilizable del código.

4) Desarrollo

La Universidad Estatal a Distancia (UNED), como parte de su proyección institucional y promoción del deporte, requiere simular la evolución de un sistema de venta de entradas para partidos de fútbol. En esta segunda etapa, se requiere incorporar persistencia en una base de datos SQL Server y habilitar la venta de entradas tanto en boletería física como en una aplicación línea.

En esta nueva fase, **el manejo de los datos se realizará mediante una base de datos SQL, sustituyendo el almacenamiento en arreglos utilizado en la primera etapa del Proyecto.** Además se implementará la **gestión de ventas de entradas en línea**, aplicando conceptos de **subprocesamiento múltiple, bases de datos, colecciones y redes**, mediante una arquitectura **cliente/servidor**. El sistema permitirá registrar transacciones de venta, actualizar el inventario en tiempo real y mantener la consistencia de los datos entre múltiples clientes conectados al servidor.

a) Requerimientos funcionales

Se recomienda implementar primero la aplicación servidor, incluyendo la conexión a la base de datos, la lógica de registro, ventas y consultas, posteriormente, se debe desarrollar la aplicación cliente, la cual enviará solicitudes al servidor mediante comunicación TCP.

El **proyecto servidor** debe cumplir con lo siguiente:

- El servidor debe escuchar y responder solicitudes de múltiples clientes TCP de forma simultánea, utilizando subprocesamiento múltiple (threads).
- Debe operar en la dirección IP 127.0.0.1 y el puerto 14500.
- Solo la aplicación servidor debe tener conexión directa con la base de datos SQL Server.
- Todas las operaciones de registro, consulta, actualización y eliminación se realizarán en el servidor.
- El servidor debe contar con una interfaz administrativa que permita visualizar en tiempo real los siguientes eventos:
 - Conexiones y desconexiones de clientes.
 - Registros de ventas procesadas.
 - Consultas y errores generados.
- Esta interfaz no mostrará los datos completos de dichas operaciones, sino únicamente los eventos asociados para fines de monitoreo.
- La bitácora debe actualizarse automáticamente conforme se reciban eventos generados por las solicitudes de los clientes.

- El servidor debe permitir al usuario administrador registrar y consultar la información de todas las entidades requeridas por el sistema.

Nota importante: A continuación, se presentan las tablas que conforman la base de datos del sistema.

Es importante destacar que, **a nivel del Modelo de Clases**, los atributos que representen relaciones entre entidades deben mantenerse como **objetos asociados**, y **no únicamente como tipos de datos primitivos**. En la base de datos, estas relaciones se representan mediante llaves foráneas; sin embargo, en las clases del sistema deben modelarse como objetos para conservar el enfoque orientado a objetos.

Esta práctica garantiza la correcta aplicación de los principios de **Programación Orientada a Objetos (POO)** y la coherencia entre el modelo de clases de la aplicación y el modelo físico de la base de datos.

i) Registrar Localidades

Permite registrar los tipos de localidad disponibles, tales como palco, preferencial, sol, sombra u otros nombres de localidad definidos por la persona usuaria.

Campos requeridos:

- IdLocalidad (int, llave primaria)
- NombreLocalidad (string)
- Precio (decimal)

Reglas de negocio:

- El IdLocalidad debe ser único.
- El NombreLocalidad no debe quedar vacío.
- El Precio debe ser mayor a cero.

Los datos deben almacenarse en la tabla Localidad.

ii) Registrar partido

Permite registrar los partidos programados para la venta de entradas.

Campos requeridos:

- IdPartido (int, llave primaria)
- Rival (string)
- Fecha (DateTime)
- Hora (string)
- Activo (bool)

Reglas de negocio:

- El IdPartido debe ser único.
- El campo Rival no debe quedar vacío.
- La Fecha del partido no debe ser menor al día actual al momento de registrar el partido.
- La Hora debe registrarse en un formato válido, por ejemplo: HH:mm.
- El campo Activo debe permitir identificar si el partido se encuentra disponible para la venta de entradas.

Los datos deben almacenarse en la tabla Partido.

iii) Registrar Vendedor

Permite registrar los vendedores que atienden la boletería física.

Campos requeridos:

- IdVendedor (int, único, llave primaria)
- Identificación (string, única)
- Nombre (string)
- Apellido (string)
- FechaNacimiento (DateTime)
- FechaIngreso (DateTime)

Reglas de negocio:

- El IdVendedor debe ser único.

- La Identificación debe ser única y no debe quedar vacía.
- Los campos Nombre y Apellido no deben quedar vacíos.
- La FechaNacimiento no debe ser mayor o igual al día actual.
- La FechaIngreso debe ser posterior a la FechaNacimiento y no debe ser mayor al día actual.

Los datos deben almacenarse en la tabla Vendedor.

iv) Registrar Cliente

Permite registrar los datos de los aficionados que compran entradas.

Campos requeridos:

- IdCliente (int, único)
- Identificación (string, única)
- Nombre (string)
- Apellido (string)
- Fecha Nacimiento
- FechaRegistro (DateTime)
- Activo (bool)

Reglas de negocio:

- El IdCliente debe ser único.
- La Identificación debe ser única y no debe quedar vacía.
- Los campos Nombre y Apellido no deben quedar vacíos.
- La FechaNacimiento no debe ser mayor o igual al día actual.
- La FechaRegistro no debe ser mayor al día actual.
- El campo Activo debe permitir identificar si el cliente se encuentra habilitado para realizar compras.

Los datos deben almacenarse en la tabla Cliente.

v) Definir localidades por partido

Permite establecer la cantidad de entradas disponibles para cada partido según la localidad seleccionada.

Campos requeridos:

- IdLocalidadPartido(int, llave primaria)
- Partido (int, llave foránea)
- Localidad (int, llave foránea)
- CantidadDisponible (int)

Los datos deben almacenarse en la tabla LocalidadPorPartido.

Reglas de negocio:

- El IdLocalidadPartido debe ser único.
- El partido debe estar previamente registrado.
- La Localidad debe estar previamente registrada.
- No se pueden registrar la misma localidad para mismo partido más de una vez.
- Para facilidad de la persona usuaria, el sistema deberá mostrar la fecha y hora del partido seleccionado al asociar la localidad.
- No se pueden registrar localidades para partidos inactivos.
- La CantidadDisponibilidad debe ser mayor a cero.

vi) Registrar venta de entrada (Boletería Física)

Permite registrar la venta de entradas desde la boletería física.

Campos requeridos:

- IdVenta (int, autogenerado, llave primaria)
- Cliente (int) (a nivel de clase objeto Cliente)
- Partido (int) (a nivel de clase objeto Partido)
- Localidad (int) (a nivel de clase objeto Localidad)
- Cantidad (int)
- Vendedor (int) (a nivel de clase objeto Vendedor)
- FechaVenta (DateTime)
- MontoTotal (decimal)
- TipoVenta (string) → En esta etapa siempre será "Boletería"

Reglas de negocio:

- El cliente debe estar previamente registrado y activo
- El vendedor debe estar previamente registrado.

- El partido debe estar previamente registrado y activo.
- La localidad debe estar previamente registrada y asociada al partido seleccionado mediante el registro de LocalidadPorPartido.
- No se pueden vender más entradas de las disponibles para la localidad seleccionada en el partido correspondiente.
- La Cantidad debe ser mayor cero.
- No se pueden vender entradas para partidos inactivos.
- El cliente debe estar activo.
- Para facilidad de la persona usuaria, deberá mostrar la fecha y hora del partido seleccionado para la venta.
- No se podrán realizar ventas de boletos para partidos con fecha menor al día actual.
- El MontoTotal debe calcularse automáticamente multiplicando la Cantidad por el Precio de la localidad seleccionada, dicho monto no es editable por las personas usuarias.
- Al realizar la venta, debe disminuir la cantidad disponible.
- Se deben manejar excepciones y validaciones en todos los procesos relacionados con la venta.
- El TipoVenta debe registrarse siempre con el valor "Boletería"

Los datos deben almacenarse en la tabla Ventas

vii) Consultar Información

El sistema debe permitir visualizar la información registrada de todas las entidades en la interfaz gráfica del servidor, utilizando un DataGridView específico para cada una de ellas.

Cada opción de consulta mostrará todos los registros almacenados en los respectivos arreglos de objetos:

- Localidades
- Partidos
- Vendedores
- Clientes
- Localidades por Partido
- Ventas

Requerimientos específicos:

- Cada entidad deberá tener una opción de consulta independiente que muestre sus datos en un DataGridView ubicado en la interfaz gráfica del servidor.
- Todas las consultas deberán obtener la información desde la base de datos SQL Server mediante la aplicación servidor.
- En los casos donde un atributo sea un objeto de otra clase, el sistema deberá mostrar también los datos relevantes de dicho objeto, no solo su referencia o identificador.
- Por ejemplo, al consultar vehículos, el campo “Localidades por partido” el sistema deberá mostrar los datos relevantes del partido y de la localidad asociada, tales como IdPartido, rival, fecha, hora, estado del partido, IdLocalidad, nombre de la localidad, precio y cantidad disponible.
- De igual forma, al consultar ventas, el sistema deberá mostrar los datos relevantes del cliente, partido, localidad, vendedor asociados a la venta, además de la cantidad vendida, fecha de venta, monto total y tipo de ventas.
- La consulta debe permitir visualizar todos los registros agregados durante la ejecución del programa, mostrando siempre los valores actualizados desde la base de datos.
- El DataGridView debe estar configurado para no permitir edición directa de datos desde la interfaz (solo lectura).
- Se recomienda ordenar las columnas de forma lógica por ejemplo: Id, nombre, atributos principales y asociaciones.

Ejemplos de visualización esperada:

Ejemplo 1 – Consulta de Localidad por Partido

Partido (IdPartido)	Partido (Rival)	Partido (Fecha)	Partido (Hora)	Partido (Activo)	Localidad (IdLocalidad)	Localidad (NombreLocalidad)	Localidad (Precio)	CantidadDisponible
1	UCR	20/06/2026	15:00	Sí	3	Palco	15000	20

Ejemplo 2 – Consulta de Venta

IdVenta	Cliente (Nombre)	Partido (Rival)	Partido (Fecha)	Partido (Hora)	Partido (Activo)	Localidad (IdLocalidad)	Localidad (NombreLocalidad)	Vendedor (Nombre)	Cantidad	FechaVenta	MontoTotal	TipVer
1000	Johan	UCR	20/06/2026	15:00	Sí	3	Palco	Juan	3	17/06/2026	45000	Bolía

El **proyecto cliente** debe cumplir con lo siguiente:

i) Conexión y Validación

- La aplicación cliente debe permitir conectarse y desconectarse del servidor a través de la red utilizando comunicación TCP.
- Al iniciar, la aplicación cliente debe solicitar el **Id o número de identificación de la persona cliente**.
- Se validará que el cliente exista y esté **activo en la base de datos SQL Server**, antes de habilitar las funcionalidades del sistema.
- Si la validación es exitosa, se mostrará el **nombre completo del cliente** y se habilitarán las opciones de compra de entradas y consultas.
- Si la validación no es exitosa, la aplicación deberá mostrar un mensaje claro a la persona usuaria y no deberá habilitar las funcionalidades de compra ni consulta.

- **Toda comunicación y validación deberá realizarse mediante solicitudes enviadas al servidor; la aplicación cliente no tendrá conexión directa a la base de datos.**

ii) Gestión de ventas

- La aplicación cliente debe permitir registrar la compra de las entradas, la cual quedará asociada al cliente que inició sesión.
- La aplicación cliente deberá enviar la solicitud de compra al servidor. El servidor será responsable de validar la información, calcular el monto total, registrar la venta en la base de datos y actualizar la disponibilidad de entradas.

Campos requeridos:

- IdVenta (int, autogenerado, llave primaria)
- Cliente (int) (a nivel de clase objeto Cliente)
- Partido (int) (a nivel de clase objeto Partido)
- Localidad (int) (a nivel de clase objeto Localidad)
- Cantidad (int)
- Vendedor (int) (a nivel de clase objeto Vendedor) (Nulo)
- FechaVenta (DateTime)
- MontoTotal (decimal)
- TipoVenta (string) → En esta etapa siempre será "En línea"

Reglas de negocio:

- El cliente debe estar previamente registrado y activo.
- El partido debe estar previamente registrado y activo.
- La localidad debe estar previamente registrada y asociada al partido seleccionado mediante el registro de LocalidadPorPartido.
- No se pueden vender más entradas de las disponibles para la localidad seleccionada en el partido correspondiente.
- La Cantidad debe ser mayor a cero.
- No se pueden vender entradas para partidos inactivos.
- Para facilidad de la persona usuaria, el sistema deberá mostrar la fecha y hora del partido seleccionado para la compra.

- Al tratarse de una venta en línea, no se registrará vendedor asociado, por lo que el campo vendedor deberá almacenarse con un “valor” nulo.
- No se podrán realizar ventas de boletos para partidos con fecha menor al día actual.
- El MontoTotal debe calcularse automáticamente multiplicando la Cantidad por el Precio de la localidad seleccionada.
- Al realizar la venta, debe disminuir la cantidad disponible.
- Se deben manejar excepciones y validaciones en todos los procesos.
- En caso de error o validaciones fallidas, se deberá mostrar un mensaje descriptivo para la persona usuaria.
- En caso de éxito, se deberá mostrar un mensaje descriptivo informando al usuario que la compra se registró correctamente.
- El servidor debe implementar mecanismos de sincronización, como semáforos o locks, para evitar conflictos de concurrencia durante las ventas simultáneas, garantizando que las operaciones eviten una sobreventa de entradas.

Los datos deben almacenarse en la tabla Ventas

iii) Consulta de Información

- La aplicación cliente debe ofrecer un módulo de consulta donde la persona usuaria pueda visualizar únicamente la información que le corresponde, específicamente las ventas registradas a su nombre, mediante controles DataGridView.
- Cada entidad tendrá su propia vista de consulta En el caso del cliente, al menos se debe incluir:
 - **Compras realizadas por el cliente autenticado.**
- La consulta a ejecutar debe limitarse exclusivamente a los datos del cliente que se encuentra actualmente autenticado. En ningún caso deberán mostrarse o exponerse datos pertenecientes a otros clientes.
- El DataGridView debe mostrar los datos relevantes de la compra y de los objetos asociados, tales como, partido, localidad, cantidad, fecha de venta, monto total y

tipo de venta. En caso de que la compra tenga un vendedor asociado, también podrá mostrarse el dato correspondiente.

- Todas las consultas deberán solicitar la información al servidor, el cual accederá a la base de datos SQL Server y devolverá los datos completos a la aplicación cliente.

Consulta de Compra

IdVenta	Partido (Rival)	Partido (Fecha)	Partido (Hora)	Partido (Activo)	Localidad (IdLocalidad)	Localidad (NombreLocalidad)	Vendedor (Nombre)	Cantidad	FechaVenta	MontoTotal	TipoVenta
1000	UCR	20/06/2026	15:00	Si	3	Palco	Juan	3	17/06/2026	45000	Boletería
3500	TEC	26/07/2026	17:00	Si	3	Palco	Sin vendedor	2	19/06/2026	30000	En Línea

b) Requerimientos técnicos

Generales

- i) El programa debe aplicar encapsulamiento, herencia y polimorfismo entre las clases del sistema.
- ii) Se deben utilizar propiedades y constructores si es necesario para inicializar y acceder a los atributos.
- iii) El manejo de errores debe realizarse con bloques try-catch, mostrando mensajes claros al usuario cuando ocurran errores comunes durante la ejecución.
- iv) La interfaz debe diseñarse en Windows Forms.
- v) Es obligatorio implementar en la solución proyectos independientes con el fin de organizar el sistema en distintas capas, tales como: Capa de Entidades, Capa de Acceso a Datos, Capa de Lógica de Negocio y Capa de Presentación (Interfaz de Usuario), entre otras que se consideren necesarias.
- vi) La aplicación debe contemplar, como mínimo, una aplicación servidor y una aplicación cliente, se recomienda utilizar un proyecto adicional de tipo biblioteca de clases para definir las entidades compartidas entre ambas aplicaciones, evitando duplicar clases en el cliente y en el servidor.
- vii) La aplicación servidor será la única que tendrá acceso directo a la base de datos SQL Server, la aplicación cliente deberá comunicarse con el servidor mediante TCP para solicitar validaciones, registrar compras y consultar información.
- viii) Cada clase debe ser creada en archivo por separado (.cs).
- ix) Las clases no deben contener métodos vacíos y se deben utilizar nombres significativos a sus variables y los métodos.
- x) Uso de comentarios, cada archivo de código (.cs) debe incluir un encabezado con comentarios que indiquen la universidad, cuatrimestre, nombre del proyecto, descripción, nombre del estudiante y fecha de desarrollo. Este encabezado debe estar presente en todas las clases y archivos del programa. Ejemplo de encabezado sugerido:

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 /* UNED III Cuatrimestre
8  * Proyecto XX: Descripción del Proyecto
9  * Estudiante:
10  * Fecha:
11  */
12
13
14 namespace CapaEntidades
15 {
16 }
```

Uso de colecciones

- xi) Se deben usar colecciones genéricas (List<>) cuando se requiera consultar de base de datos y retornar más de un registro.
- xii) Las colecciones genéricas se deben utilizar como “DataSource” en los controles “combobox” y “DataGridView”, según corresponda.
- xiii) No se debe utilizar “DataSet” ni “DataTable” para la carga o presentación de la información.

Base de datos

- xiv) Se debe almacenar y consultar la información en una base de datos SQL Server. No se permite el uso de otro motor de base de datos.
- xv) Para la cadena de conexión se debe utilizar seguridad integrada de Windows y definirla únicamente en un archivo de configuración “App.config”.
- xvi) El script de creación de la base de datos se publicará en la plataforma AprendeU. El estudiante debe utilizar dicho script para crear la base de datos y no debe modificar su estructura.

Redes

- xvii) La solución será probada utilizando al menos 5 aplicaciones cliente de forma simultánea y una única instancia de la aplicación servidor.
- xviii) La aplicación servidor debe mantenerse activa durante la ejecución de las aplicaciones cliente y debe registrar en la bitácora los eventos de conexión, desconexión, consultas, compras, ventas procesadas y errores generados.

Manual

xix) Debe presentar un manual breve con los pasos necesarios para probar la solución entregada. Debe contener:

- (1) Imágenes de las pantallas de las soluciones.
- (2) Portada: nombre del estudiante, curso, cuatrimestre, número de grupo, tutor asignado, nombre Proyecto).
- (3) Requisitos para la ejecución de la aplicación
- (4) Pasos detallados para instalar y ejecutar primero la aplicación Servidor y luego la aplicación Cliente.
- (5) Instrucciones de prueba de las funcionalidades principales: conexión de usuarios, compra de boletos y consultas.
- (6) Capturas de pantalla principales de la solución, tanto del servidor como del cliente.
- (7) El Manual debe entregarse en formato PDF, con el nombre; NombreApellido-ManuaP2.pdf

Diagrama cliente/servidor

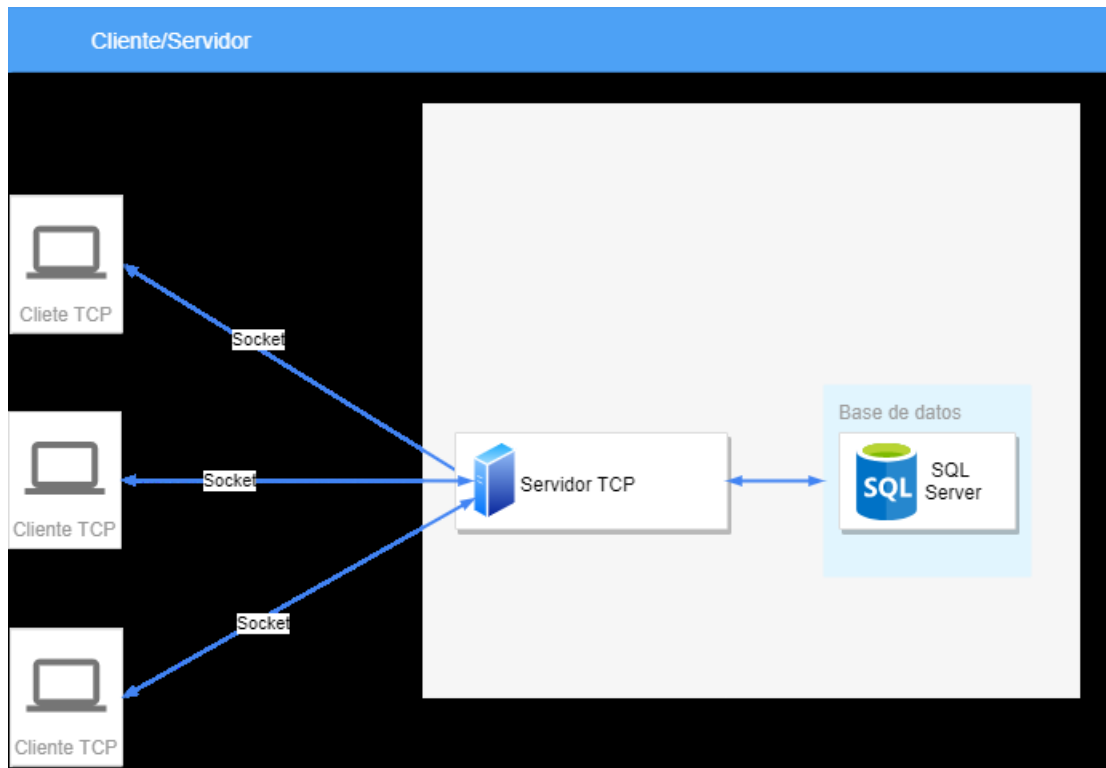
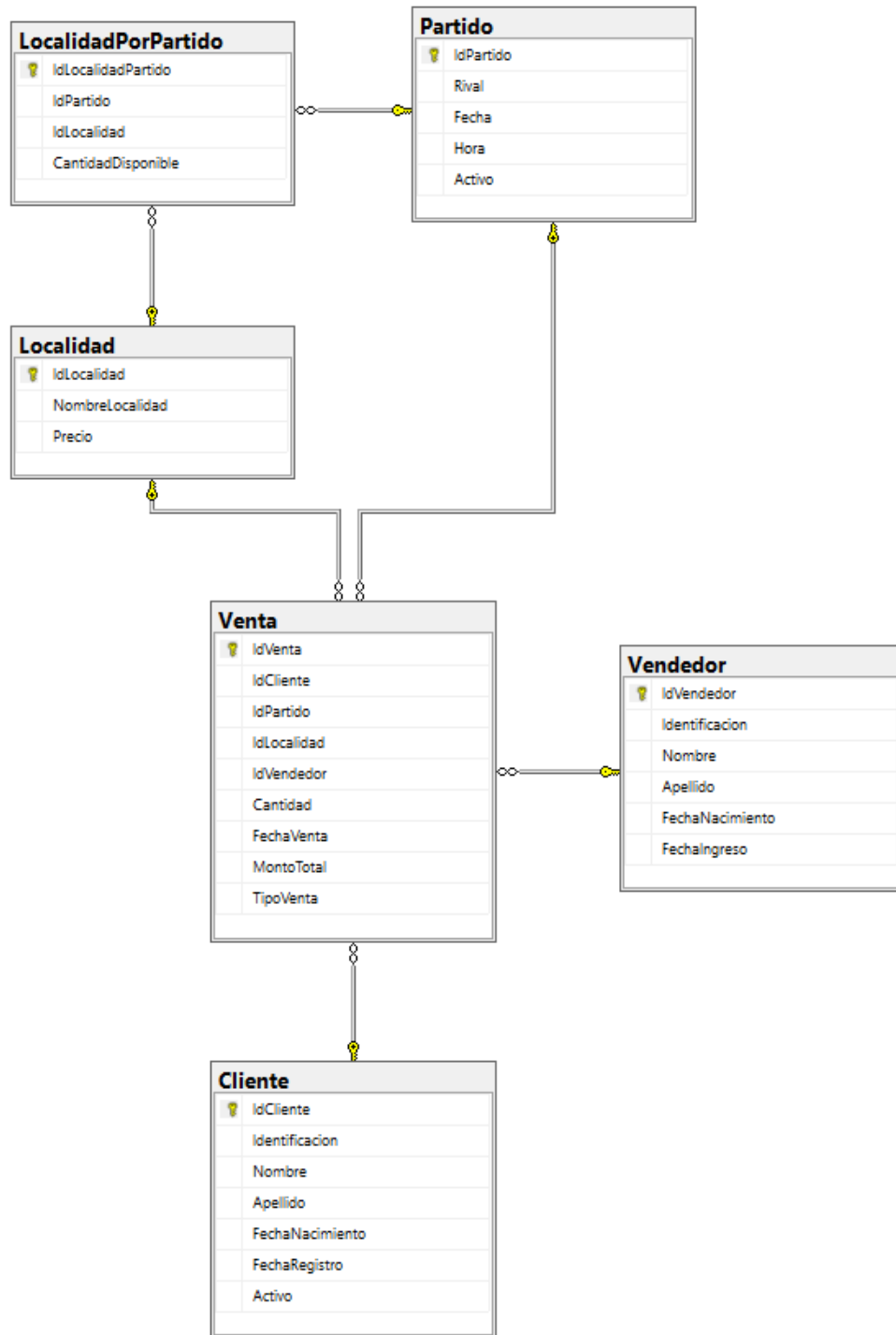


Diagrama base de datos



5) Honestidad Académica



<https://audiovisuales.uned.ac.cr/play/player/23048>

Nota importante

- Cada estudiante es responsable del contenido que entrega.
- Si el archivo enviado no corresponde al proyecto correcto, no se aceptarán entregas posteriores fuera de la fecha establecida.
- Si se detecta que el contenido del archivo coincide con otro estudiante o no es de su autoría, se aplicarán las sanciones indicadas en el documento de Lineamientos ante casos de plagio disponible en la plataforma.
- En caso de utilizar herramientas de Inteligencia Artificial Generativa para apoyar el desarrollo del proyecto el estudiante debe:
 - Indicarlo explícitamente en la documentación del proyecto (comentarios en el código y/o una breve sección en el documento o carta adjunta).
 - Revisar, adaptar y comprender todo el código generado.
 - Asumir la responsabilidad total sobre el funcionamiento del sistema.
 - Indicar cuáles herramientas utilizó.
 - Adjuntar a la documentación los Prompt(s) principales que utilizó y cuyo resultado fue incorporado, total o parcial, en el proyecto.
 - El uso de IA generativa para entregar un sistema que el estudiante no comprende o no pueda explicar podrá considerarse una falta a la honestidad académica y, según la gravedad, tratarse como plagio, de acuerdo con los Lineamientos ante casos de plagio y la normativa vigente de la UNED.

6) Entregables

- **Es obligatorio que incluya todo el directorio donde se encuentra el Proyecto # 2.**
- El **Proyecto #2** debe estar desarrollado en **Visual Studio Community 2026** que es la herramienta oficial del curso.
- Se recomienda crear un tercer proyecto de entidades tipo biblioteca de clases para las entidades a utilizar en la solución, de esa manera no tendrá que duplicar las clases de entidades en el proyecto cliente y servidor, en estos solo se agrega la referencia o dll.
- El programa debe ser modular, utilizando de la mejor manera funciones definidas por usted.
- **Los trabajos deben realizarse en forma individual. Dentro del código del programa debe de indicar la documentación que explique cómo fue realizado el programa.**
- **Si utiliza código de algún ejemplo del libro, del algún profesor o de otra fuente que no sea de su autoría, debe de indicarlo.**
- Comprima todos los archivos en un solo archivo .zip o .rar.
- **Nombre del archivo que envía:** debe ser nombre y primer apellido del estudiante, y nombre de la tarea. **Ejemplo: JuanRojas-proyecto2.**
- La entrega del proyecto las fechas establecidas en la plataforma de aprendizaje en línea Moodle en el apartado que se indique.
- Si no concluyó a tiempo la tarea, debe entregar lo que pudo hacer e incluir una carta explicando las razones por las cuales no finalizó.



PROYECTOS PROGRAMADOS

ANTES DE ENVIAR EL PROYECTO

1. Asegúrese de que el programa **compile** y se ejecute correctamente.
2. Revise el archivo comprimido (.zip) antes de enviarlo para confirmar que incluye **todos los archivos necesarios** y que el proyecto funciona sin errores.
3. Si el proyecto **compila pero no se ejecuta correctamente**, o si **no compila**, **se evaluará según los criterios establecidos en la rúbrica**.
4. Si la calificación final, según la rúbrica, es 0 (cero), **no podrá realizar la defensa oral del proyecto**.
5. Si no entrega el proyecto (archivo .zip), su calificación será 0 (cero), salvo en **casos de fuerza mayor debidamente justificados** y tramitados conforme al Reglamento General Estudiantil.

7) Rúbrica de Evaluación

Criterio	Excelente	Aceptable	Insuficiente	Nulo
Estructura y presentación del código fuente.	Presenta un código claro, ordenado y legible, acorde con las normas del IDE y las buenas prácticas de programación. Utiliza nombres significativos, mantiene una estructura coherente y facilita la comprensión de la solución. 5 puntos	Presenta un código funcional y comprensible, aunque con omisiones menores en orden, nombres utilizados, estructura general o presentación del código. 3 puntos	Presenta un código desordenado, confuso o poco claro, con errores importantes en la presentación, estructura, nombres utilizados o claridad general de la solución. 1 punto	No presenta lo solicitado, el código es ilegible o no permite identificar una estructura mínima funcional. 0 puntos

Criterio	Excelente	Aceptable	Insuficiente	Nulo
Registra localidades y partidos en el servidor.	Registra correctamente localidades y partidos desde la aplicación servidor, aplicando las validaciones, restricciones y reglas necesarias según el enunciado. Mantiene coherencia en el almacenamiento o y gestión de la información en la base de datos. 5 puntos	Registra localidades y partidos de forma funcional, aplicando la mayoría de las validaciones, restricciones y reglas necesarias, aunque presenta omisiones menores en el almacenamiento o gestión de la información. 3 puntos	Registra localidades y partidos de forma incompleta o con errores importantes en las validaciones, restricciones, reglas o almacenamiento de la información. 1 punto	No registra localidades ni partidos, o la funcionalidad presentada no corresponde con lo solicitado. 0 puntos

Criterio	Excelente	Aceptable	Insuficiente	Nulo
Registra vendedores y clientes en el servidor.	Registra correctamente vendedores y clientes desde la aplicación servidor, aplicando las validaciones, restricciones y reglas necesarias según el enunciado. Mantiene coherencia en el almacenamiento o y gestión de la información en la base de datos. 5 puntos	Registra vendedores y clientes de forma funcional, aplicando la mayoría de las validaciones, restricciones y reglas necesarias, aunque presenta omisiones menores en el almacenamiento o gestión de la información. 3 puntos	Registra vendedores y clientes de forma incompleta o con errores importantes en las validaciones, restricciones, reglas o almacenamiento de la información. 1 punto	No registra vendedores ni clientes, o la funcionalidad presentada no corresponde con lo solicitado. 0 puntos

Criterio	Excelente	Aceptable	Insuficiente	Nulo
Registra localidades por partido en el servidor.	Registra correctamente localidades por partido desde la aplicación servidor, aplicando las validaciones, restricciones y reglas necesarias según el enunciado. Mantiene coherencia en la asociación de datos, disponibilidad y almacenamiento de la información. 10 puntos	Registra localidades por partido de forma funcional, aplicando la mayoría de las validaciones, restricciones y reglas necesarias, aunque presenta omisiones menores en la asociación, disponibilidad o almacenamiento de la información. 7 puntos	Registra localidades por partido de forma incompleta o con errores importantes en las validaciones, restricciones, asociación de datos, disponibilidad o almacenamiento. 3 puntos	No registra localidades por partido, o la funcionalidad presentada no corresponde con lo solicitado. 0 puntos

Criterio	Excelente	Aceptable	Insuficiente	Nulo
Consulta localidades, partidos, vendedores, clientes, localidades por partido y ventas en el servidor.	Consulta correctamente desde la aplicación servidor la información requerida según el enunciado. Presenta los datos de forma clara, actualizada y coherente, incluyendo la información relevante de las entidades asociadas cuando corresponda. 10 puntos	Presenta las consultas en el servidor de forma funcional, aunque con omisiones menores en datos mostrados, formato, actualización, relaciones entre entidades o configuración del DataGridView. 7 puntos	Presenta consultas incompletas, confusas o con errores importantes en la información mostrada, actualización de datos o representación de entidades asociadas. 3 puntos	No implementa las consultas en el servidor, o la información mostrada no corresponde con lo solicitado. 0 puntos

Criterio	Excelente	Aceptable	Insuficiente	Nulo
Conexión de la aplicación cliente con el servidor mediante TCP y administración de conexiones en el servidor.	Implementa correctamente la comunicación entre la aplicación cliente y el servidor mediante TCP, respetando las condiciones establecidas en el enunciado. El servidor administra adecuadamente las conexiones y desconexiones de múltiples clientes. 10 puntos	La comunicación cliente-servidor funciona de manera general, aunque presenta omisiones menores en conexión, desconexión, administración de clientes, mensajes o control de estado. 7 puntos	La comunicación TCP es limitada, inestable o incompleta, con errores importantes en el intercambio de información o en la administración de conexiones. 3 puntos	No implementa conexión TCP entre cliente y servidor, o la comunicación no funciona. 0 puntos

Criterio	Excelente	Aceptable	Insuficiente	Nulo
Validación del usuario cliente en la aplicación cliente.	Valida correctamente al cliente mediante solicitudes enviadas al servidor, aplicando las restricciones necesarias antes de habilitar las funcionalidades de compra y consulta. Mantiene coherencia con la información almacenada en la base de datos. 5 puntos	Valida al cliente de forma funcional, aunque presenta omisiones menores en mensajes, control de estado, habilitación de opciones o comunicación con el servidor. 3 puntos	La validación del cliente es incompleta o débil, permitiendo inconsistencias en el acceso, estado del cliente o comunicación con el servidor. 1 punto	No valida el usuario cliente, o la validación no se realiza mediante el servidor. 0 puntos

Criterio	Excelente	Aceptable	Insuficiente	Nulo
Realiza compra de boletos en la aplicación cliente.	Permite registrar correctamente compras de boletos desde la aplicación cliente, enviando la solicitud al servidor y aplicando las validaciones, restricciones y reglas necesarias según el enunciado. Mantiene coherencia en el registro, cálculo, almacenamiento o y actualización de la disponibilidad. 10 puntos	Realiza compras de boletos de forma funcional, aplicando la mayoría de las validaciones, restricciones y reglas necesarias, aunque presenta omisiones menores en cálculo, almacenamiento, actualización o comunicación con el servidor. 7 puntos	Realiza compras de boletos de forma incompleta o con errores importantes en validaciones, cálculo, disponibilidad, almacenamiento o comunicación cliente-servidor. 3 puntos	No implementa la compra de boletos desde la aplicación cliente, o la funcionalidad no corresponde con lo solicitado. 0 puntos

Criterio	Excelente	Aceptable	Insuficiente	Nulo
Consulta compras de boletos en la aplicación cliente.	Consulta correctamente las ventas registradas a nombre del cliente autenticado, es decir, sus compras realizadas. Muestra la información requerida de forma clara, actualizada y coherente, sin exponer datos de otros clientes. 10 puntos	Consulta las compras de forma funcional, aunque presenta omisiones menores en datos mostrados, formato, actualización, mensajes al usuario o filtrado de información. 7 puntos	Consulta las compras de forma incompleta o confusa, con errores importantes en la información mostrada, actualización de datos o filtrado por cliente autenticado. 3 puntos	No implementa la consulta de compras del cliente, o expone información de otros clientes. 0 puntos

Criterio	Excelente	Aceptable	Insuficiente	Nulo
Muestra eventos enviados por los clientes conectados en la aplicación servidor.	Muestra correctamente en la aplicación servidor los eventos generados por los clientes conectados, de acuerdo con lo solicitado en el enunciado. La bitácora se actualiza de forma clara y oportuna. 5 puntos	Muestra los eventos de forma funcional, aunque presenta omisiones menores en mensajes, actualización, claridad o registro de algunos eventos. 3 puntos	La visualización de eventos es incompleta o confusa, con errores importantes en el registro, actualización o claridad de la bitácora. 1 punto	No muestra eventos en la aplicación servidor, o la bitácora no funciona. 0 puntos

Criterio	Excelente	Aceptable	Insuficiente	Nulo
Correcto uso de subprocesamiento o múltiple en el servidor.	Implementa correctamente el subprocesamiento múltiple en el servidor para atender clientes de forma simultánea, manteniendo la estabilidad y fluidez de la aplicación. 5 puntos	Implementa subprocesamiento o múltiple de forma funcional, aunque presenta omisiones menores en la administración de hilos, cierre de conexiones o respuesta del servidor. 3 puntos	El subprocesamiento múltiple es limitado o incorrecto, afectando la atención simultánea de clientes, la estabilidad o la respuesta del servidor. 1 punto	No implementa subprocesamiento múltiple en el servidor. 0 puntos

Criterio	Excelente	Aceptable	Insuficiente	Nulo
Implementa un mecanismo para administrar la concurrencia en el servidor al realizar una venta y evitar la sobreventa.	Implementa correctamente un mecanismo de control de concurrencia que protege el proceso de venta y mantiene la coherencia de la disponibilidad durante operaciones simultáneas. Evita la sobreventa de entradas. 5 puntos	Implementa un mecanismo de concurrencia funcional, aunque presenta omisiones menores en la protección del proceso de venta, actualización de disponibilidad o control de errores. 3 puntos	El mecanismo de concurrencia es débil o incompleto, con riesgo de inconsistencias en la disponibilidad o en las ventas simultáneas. 1 punto	No implementa un mecanismo de concurrencia, o el sistema permite sobreventa de entradas. 0 puntos

Criterio	Excelente	Aceptable	Insuficiente	Nulo
Correcto uso del manejo de excepciones.	Aplica correctamente el manejo de excepciones y las validaciones necesarias según el enunciado, mostrando mensajes claros para orientar a la persona usuaria y manteniendo la estabilidad de la solución durante la ejecución. 5 puntos	Aplica el manejo de excepciones y las validaciones de forma funcional, aunque presenta omisiones menores en algunos procesos, mensajes o controles requeridos. 3 puntos	Aplica el manejo de excepciones de forma limitada o incorrecta. Presenta errores importantes en el control de validaciones, mensajes al usuario o estabilidad de la solución. 1 punto	No implementa manejo de excepciones ni validaciones relevantes para la solución. 0 puntos

Criterio	Excelente	Aceptable	Insuficiente	Nulo
Interfaz de usuario es fácil de usarse y presenta buen diseño.	Presenta interfaces en Windows Forms claras, ordenadas e intuitivas, que facilitan el uso de la solución y permiten realizar los procesos solicitados de forma coherente. Utiliza controles adecuados, mensajes orientadores y una distribución visual acorde con las funcionalidades requeridas. 5 puntos	Presenta interfaces funcionales en Windows Forms, aunque con omisiones menores en orden, claridad, distribución de controles, mensajes al usuario o facilidad de uso. 3 puntos	Presenta interfaces confusas, desordenadas o poco intuitivas, que dificultan el uso de la solución. Omite elementos importantes solicitados en el enunciado o presenta controles poco adecuados. 1 punto	No presenta interfaces funcionales en Windows Forms, o las interfaces impiden la interacción adecuada con la solución. 0 puntos

Criterio	Excelente	Aceptable	Insuficiente	Nulo
Manual de uso de la solución.	Presenta un manual completo, claro y ordenado, con la información necesaria para instalar, ejecutar y probar la solución según lo solicitado en el enunciado. Incluye evidencias visuales pertinentes del servidor y del cliente. 5 puntos	Presenta un manual funcional, aunque con omisiones menores en requisitos, pasos, capturas, instrucciones de prueba o formato solicitado. 3 puntos	Presenta un manual incompleto o confuso, con omisiones importantes que dificultan ejecutar o probar correctamente la solución. 1 punto	No presenta manual, o el documento entregado no permite comprender cómo ejecutar y probar la solución. 0 puntos
Total	100 puntos			

Uso responsable de Inteligencia Artificial Generativa (IAG)

De conformidad con el Lineamiento para el uso de la Inteligencia Artificial y la Política Institucional de Inteligencia Artificial en la UNED, la persona estudiante podrá utilizar herramientas de Inteligencia Artificial (IA) únicamente como apoyo para el proceso de aprendizaje, siempre que su uso sea ético, crítico, transparente y bajo supervisión humana.

La IA puede emplearse para aclarar conceptos, organizar ideas, revisar redacción, analizar errores, depurar código o mejorar la presentación del proyecto. Sin embargo, no sustituye el análisis, diseño, programación, documentación, pruebas ni defensa del proyecto. El trabajo presentado debe evidenciar comprensión, criterio técnico y autoría académica de la persona estudiante.

Cuando se utilicen herramientas de IA, su uso deberá declararse expresamente en el documento del proyecto, indicando la herramienta empleada, el propósito del uso, las partes del trabajo donde se aplicó y la revisión, corrección o adaptación realizada por la persona estudiante. Esta declaración responde al principio de transparencia establecido en los lineamientos institucionales para el uso de IA.

No se permite presentar como propio contenido generado por IA sin revisión, comprensión, adaptación y declaración de uso. Tampoco se debe ingresar en herramientas de IA información sensible, datos personales, claves de acceso, credenciales, información institucional confidencial o datos de terceros.

El uso indebido de IA, la omisión de su declaración, la presentación de contenido generado por IA como propio, el plagio, la falsificación, la alteración o cualquier acción que afecte la integridad académica podrá ser valorada conforme a la normativa institucional vigente.